CS185 Homework 4 Report

Q-Learning and Actor-Critic Algorithms

Name Of Student: Chong Wei Kee

SID: 3041940996

1. **Approximate KL.** In a short paragraph, explain why the estimator

$$e^{\Delta} - \Delta - 1$$

is a valid sampled-token estimator for the KL term used in this assignment, and why computing the exact full-vocabulary KL at every token position would be much more expensive in both compute and memory.

This is a valid sampled-token estimator for the KL term as the expected value of the expression is equivalent to the KL divergence between the current policy and the reference policy. Computing the exact full-vocabulary KL at every token position would scale the cost with the size of the model's vocabulary. LLM vocabulary sizes could range from 30,000 to 100,000 so this adds a significant constant time factor to compute and memory as well as the size of the tensor of log probs would be of shape (B,L,V) rather than (B,L).

2. **Implementation.** Briefly describe the order in which you implemented the TODOs and one bug or confusion point that you had to resolve.

I followed the order that the homework document suggested. Specifically,

4.1 Recommended Implementation Order

Implement the TODOs in this order:

1. `compute_per_token_logprobs`, `build_completion_mask`, and `approx_kl_from_logprobs`
2. `iter_minibatches`
3. `compute_group_advantages` and `maybe_normalize_advantages`
4. `Reinforce.update`
5. `GRPO.update`

Bug 1) In compute_group_advantages, I initially normalised the advantages as follows:

```
for g in range(advantages.shape[0]):
    r_g = advantages[g]
    mean_g = r_g.mean()
    std_g = r_g.std(unbiased=False)
    # if std_g < 1e-8:
    #     # If the std is too small, we can end up with huge advantages that destabilize training.
    #     # In this case, we can skip normalization and just use zero-centered rewards as advantages for this group.
    #     advantages[g] = r_g - mean_g
    # else:
    advantages[g] = (r_g - mean_g) / (std_g + eps)
```

However, this caused the train reward to not converge as the expected trend stated. After changing it to

```
mean_reards = advantages.mean(dim=1, keepdim=True)
std_rewards = advantages.std(dim=1, keepdim=True, unbiased=False)
advantages = (advantages - mean_reards) / (std_rewards + eps)
```

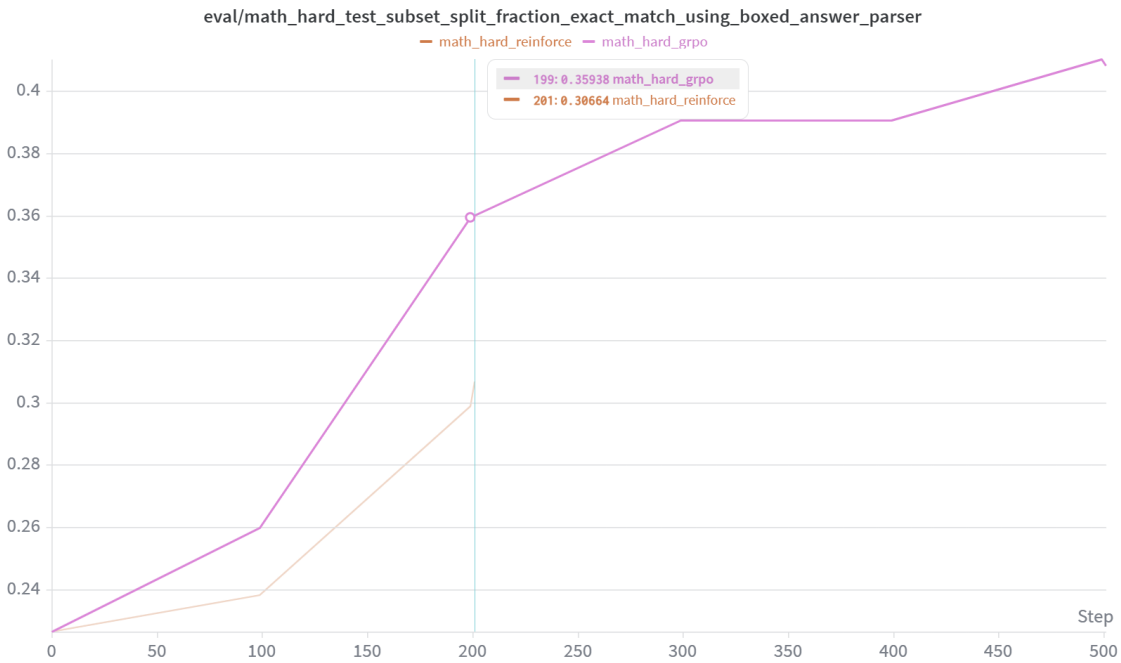
This problem was resolved. I suspect changing the code affected how the GPU normalised each group and the parallel processing might have caused my initial code to not run as I expected.

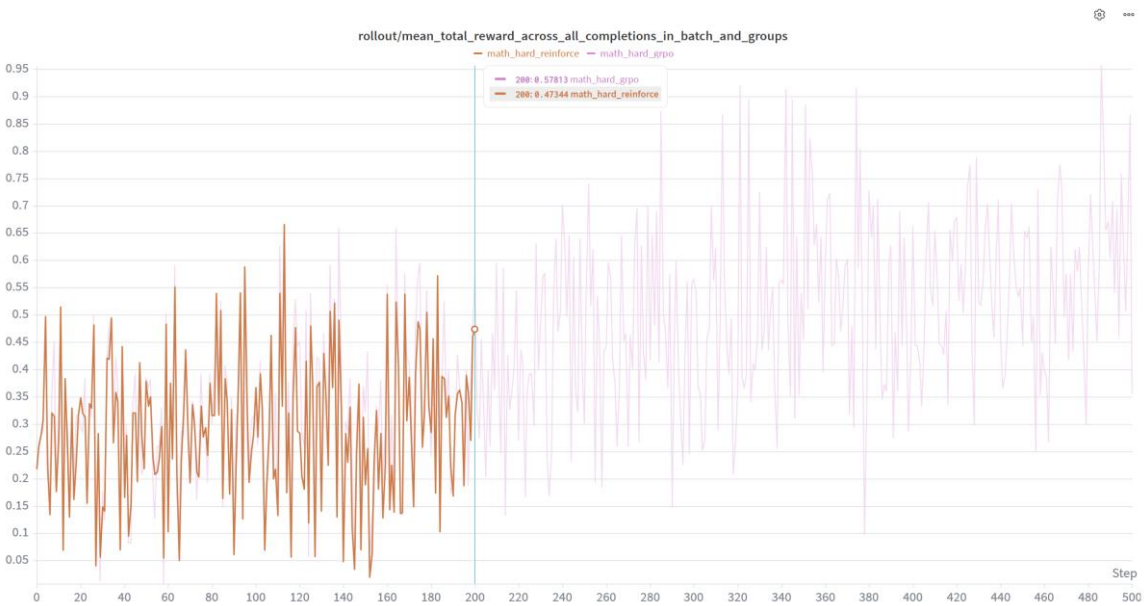
Bug 2)

```
# pg_loss = -(adv * seq_logp).mean()  # [B_mb] -> scalar
pg_loss = -masked_mean(adv * seq_logp, torch.ones_like(adv), eps=1e-8)
```

I initially implemented pg_loss in grpo and reinforce by simply calling .mean(). This caused my eval rewards to be lower than expected. I learned that using masked mean was necessary due to the padding tokens which would be accounted for without the mask as well as the eps in the calculation of mean which would affect the scaling of the loss and potentially change gradient updates.

3. **GR-REINFORCE vs. GRPO on math.** Compare the WandB curves for the math runs. How do the two methods differ over the first 200 iterations? Why is that comparison interesting given the way the provided commands were chosen?

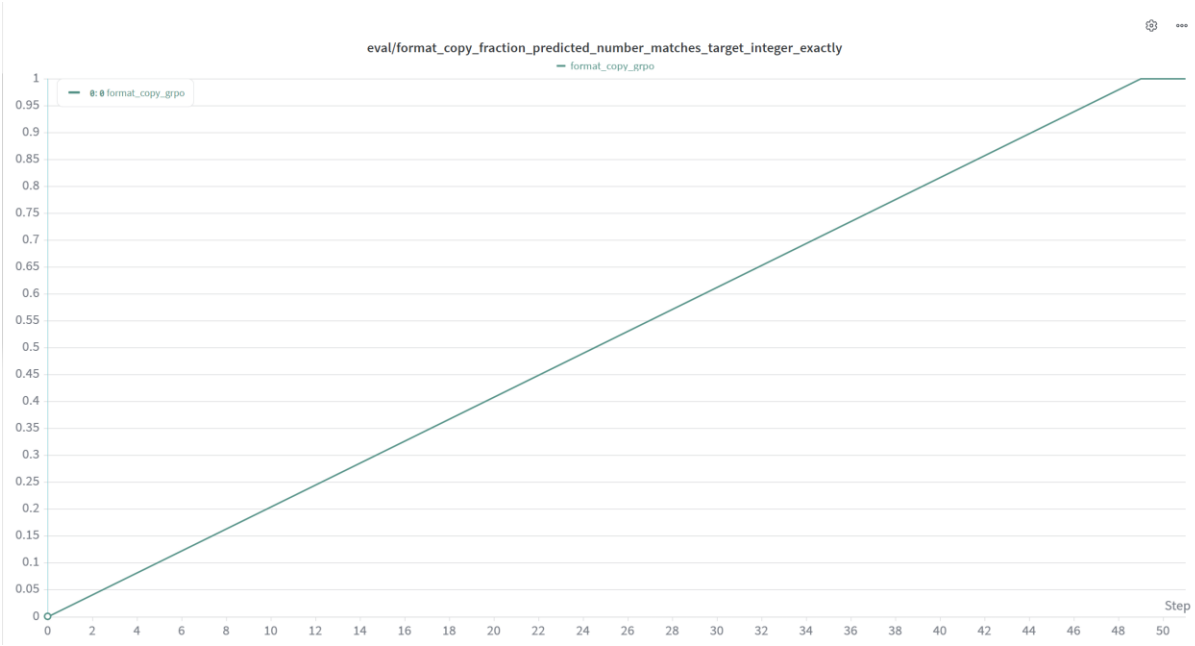


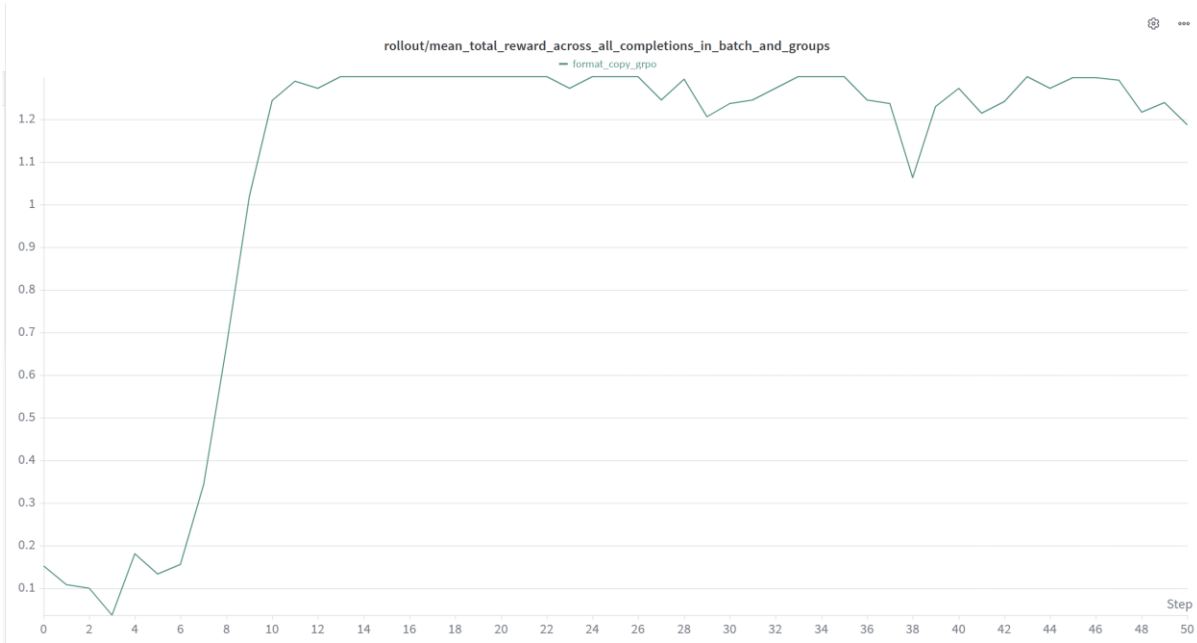


GRPO achieves a higher training and eval reward over the first 200 iterations. Given that GRPO performs 2 clipped PPO optimiser updates per rollout batch while GR-REINFORCE is strictly on policy and one-pass, GRPO would have more stable gradient updates and more updates for the same number of batches. This is interesting because although GR-REINFORCE is strictly using on policy data while GR-REINFORCE "reuses" data for multiple updates to be more sample efficient, GRPO shows improved optimisation.

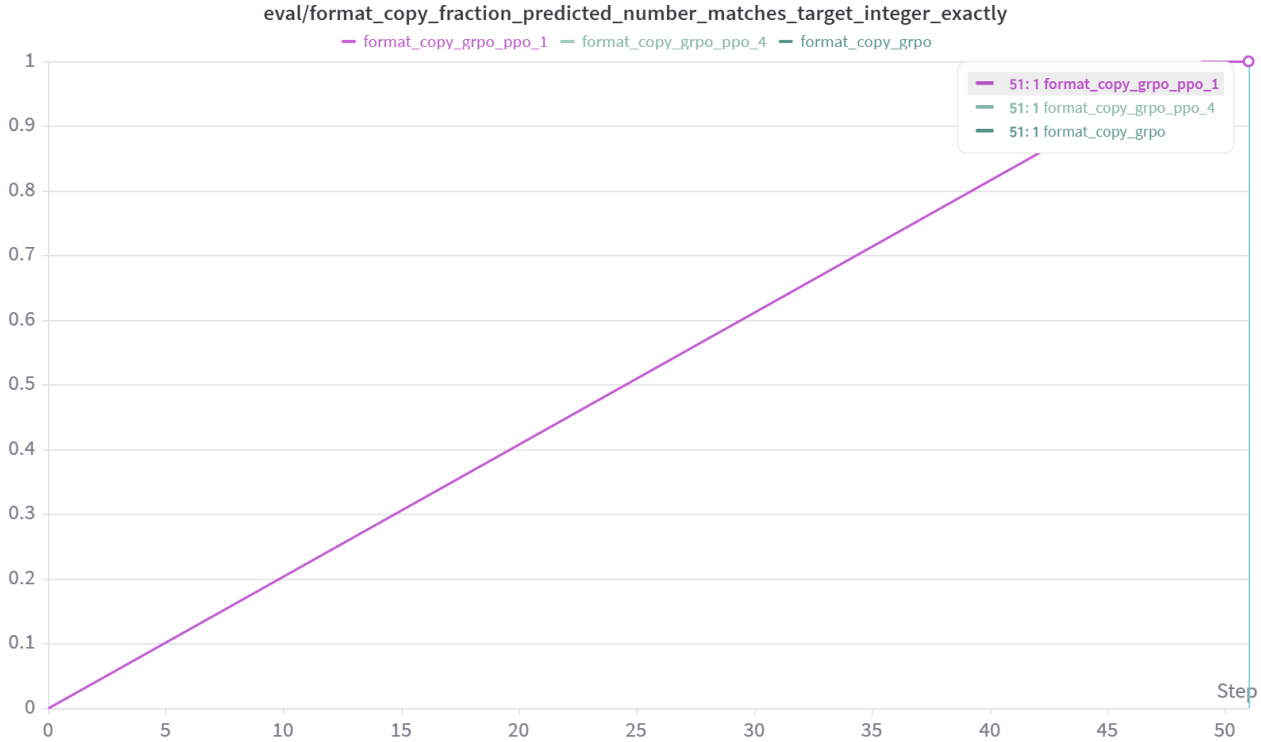
4. **GRPO ablations on `format_copy`.** Summarize the extra GRPO runs you tried. Which hyperparameters mattered most? Which settings made learning worse, and why do you think they did?

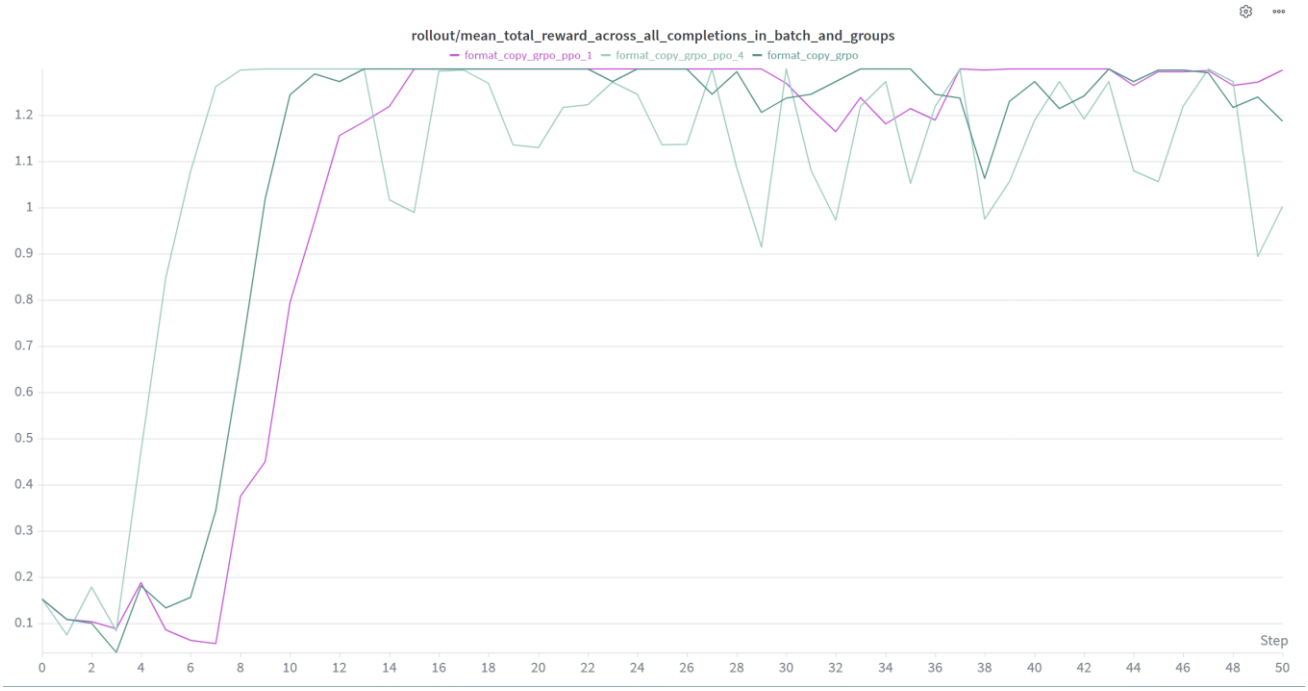
Original format_copy grpo curves (ppo_epochs = 2, kl_coef = 0.05,clip_eps = 0.2, minibatch_size = 8, grad_accum_steps = 6)

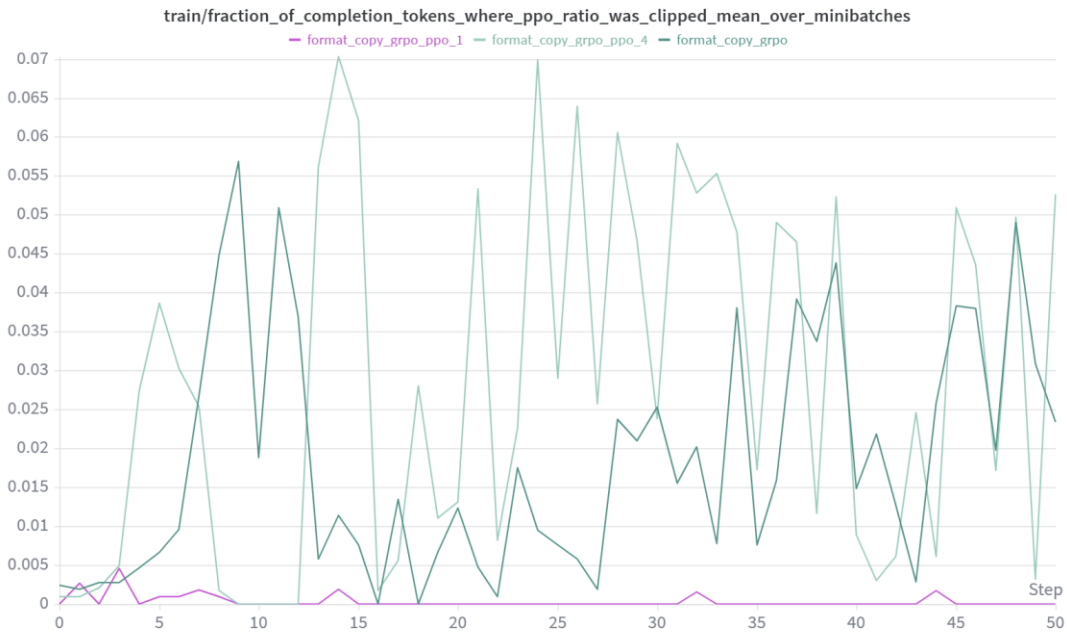


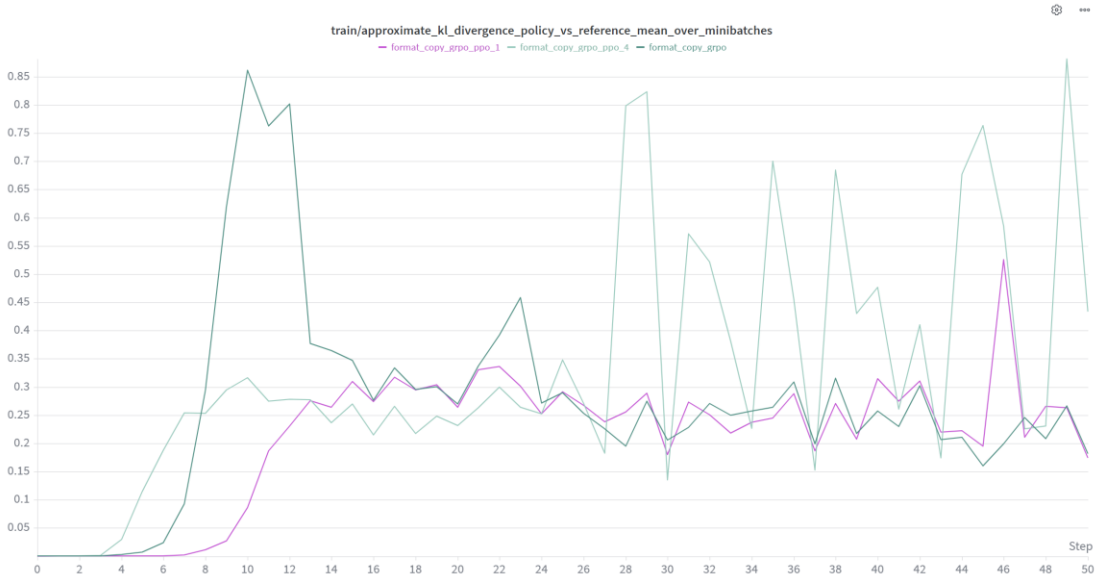


PPO Epochs Ablation







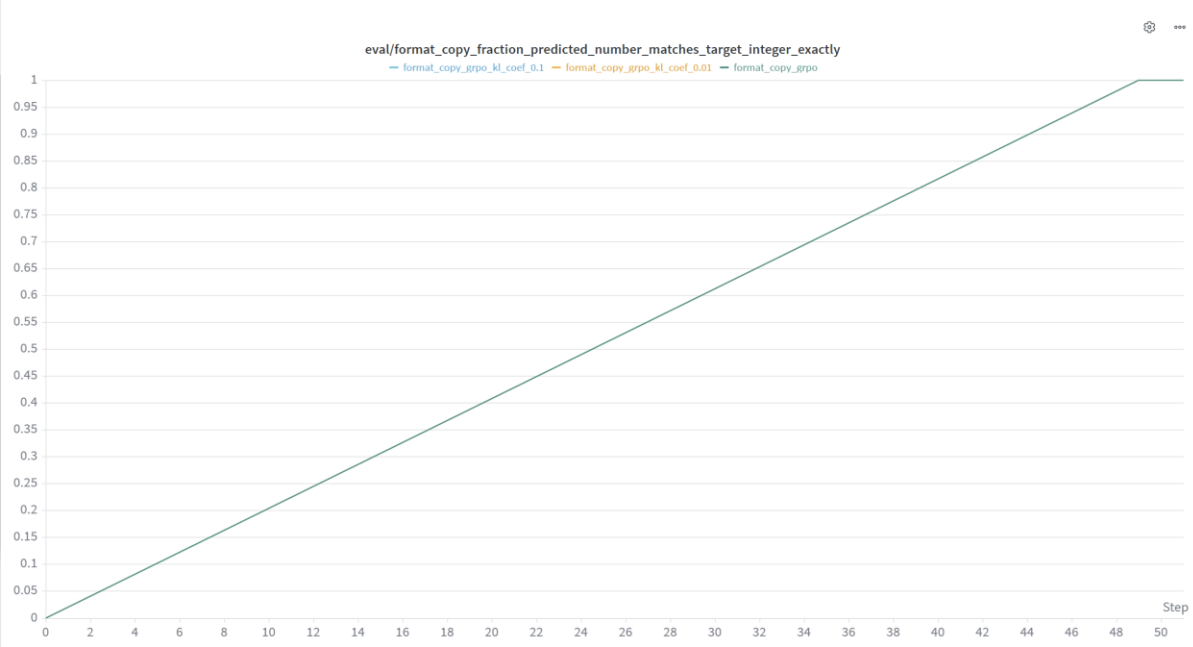


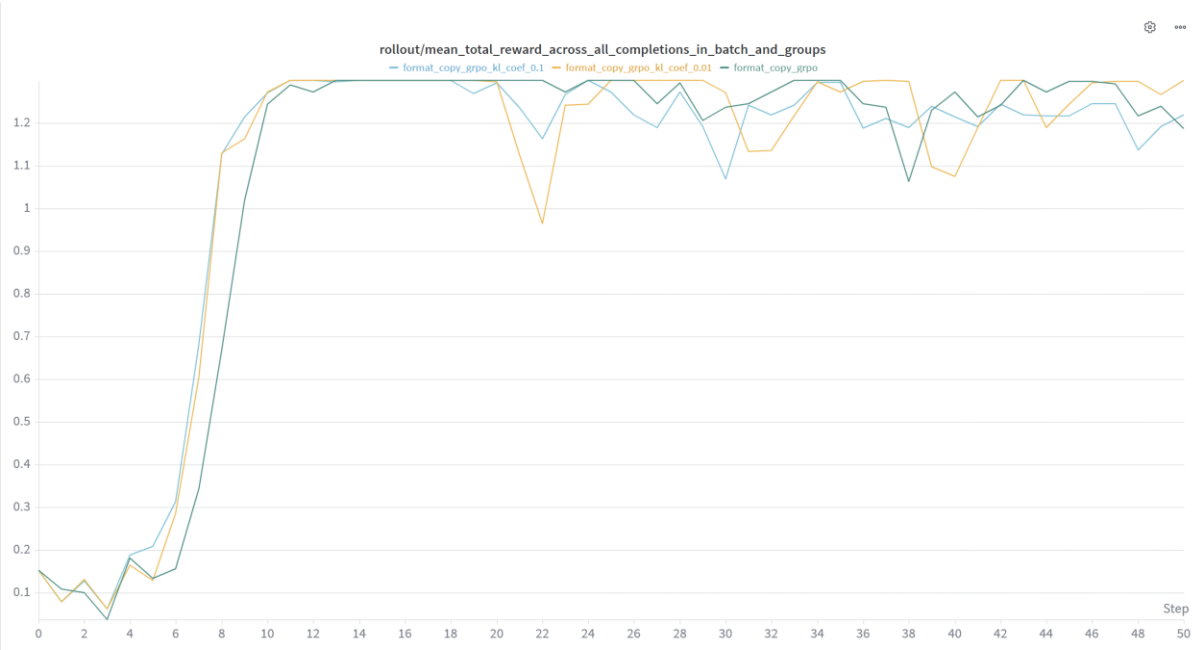
I ran format_copy grpo 2 other times, keeping other hyperparameters the same but changing ppo_epochs to 1 and 4. This controls the number of clipped PPO updates taken for one rollout. It seems both other runs saw improved train rewards generally though eval rewards maintained the same trend.

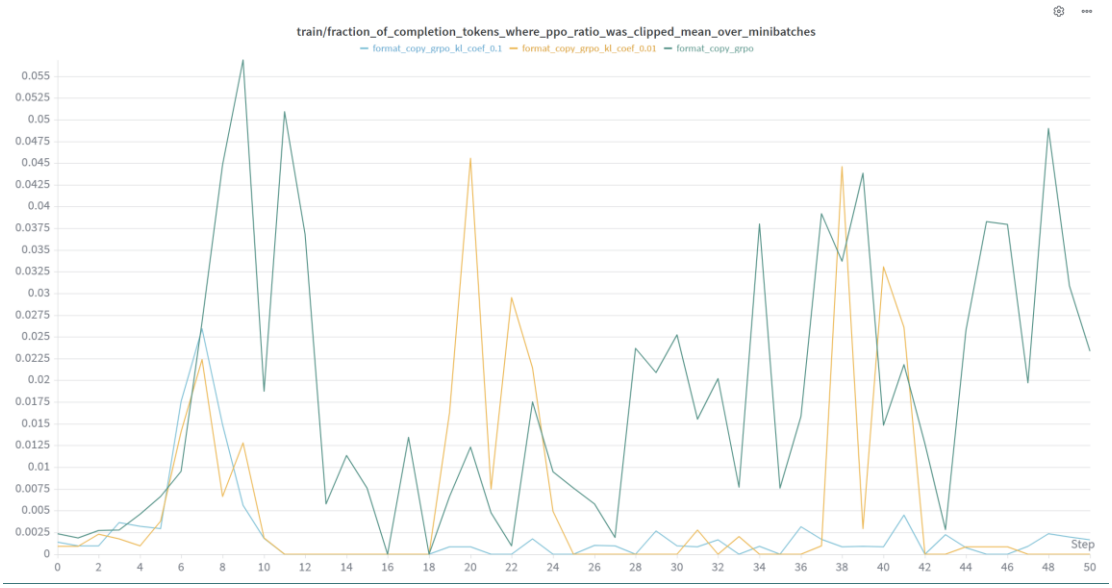
The low fraction of clipped completion tokens for ppo_epochs = 1 can be explained due to the ratio of Pi_current to Pi_old not being able to drift that drastically if only one PPO update is taken per rollout, whereas for ppo_epochs = 2 or ppo_epochs = 4, Pi_current might change greatly after multiple PPO updates on stale data, compared to Pi_old

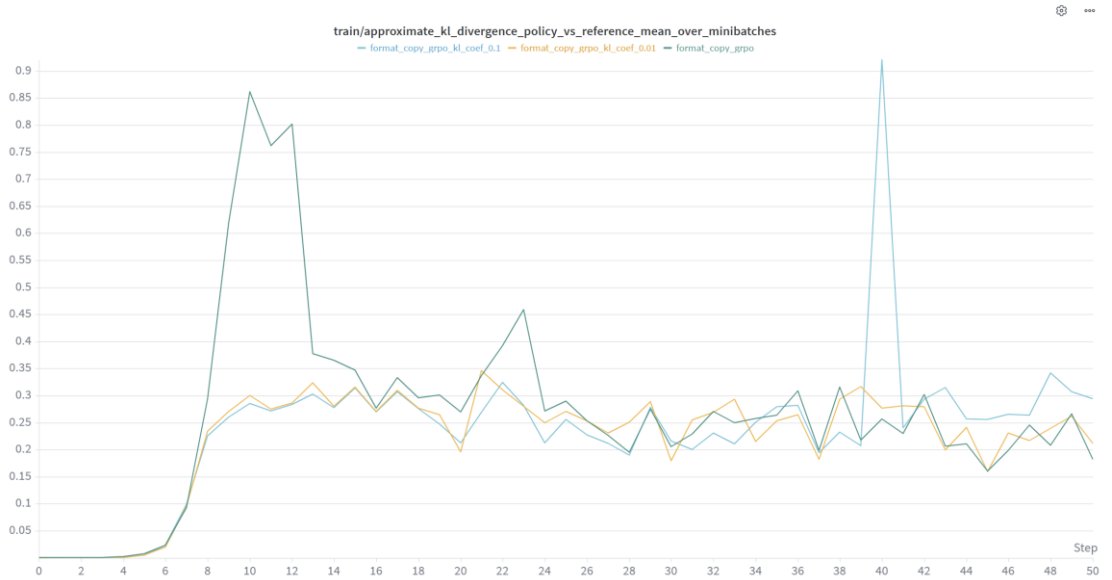
The higher variance of KL_divergence for the run with ppo_epochs = 4 makes sense because more PPO updates are taken for the same data which might cause the policy to overfit to stale data.

KL Coefficient Ablation







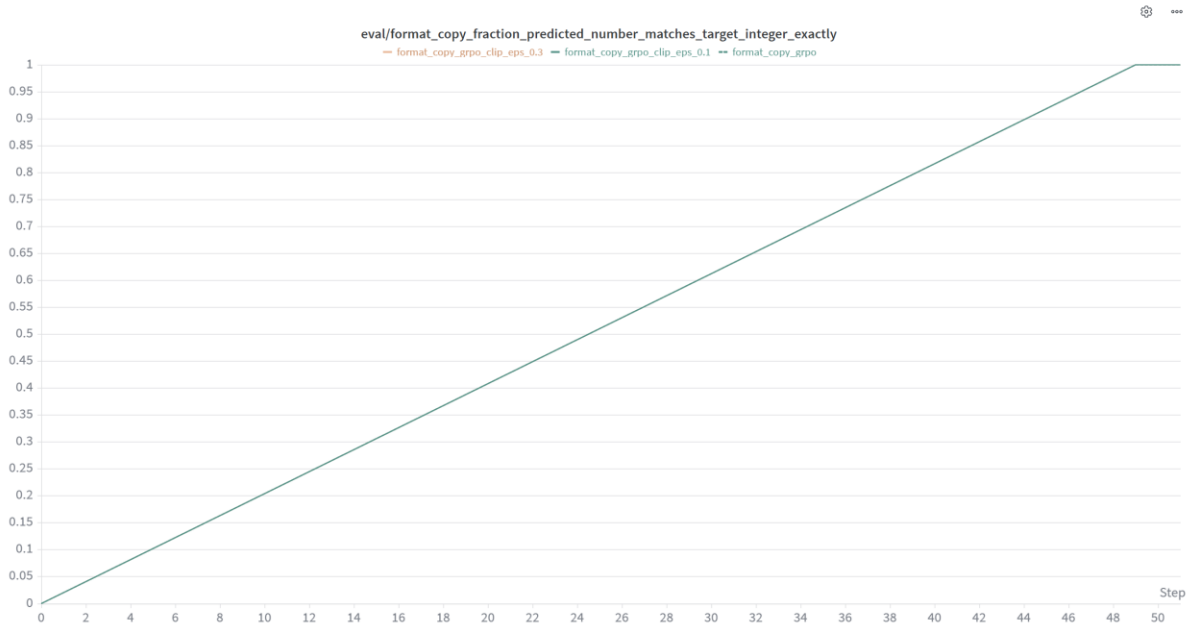


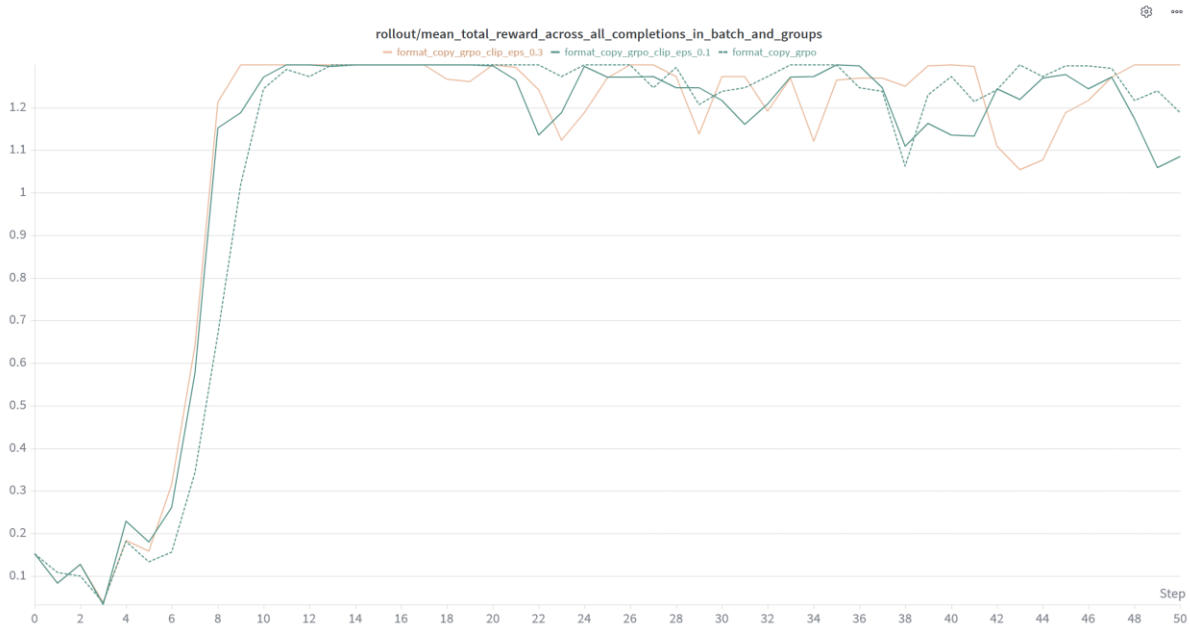
I ran format_copy grpo 2 other times, keeping other hyperparameters the same but changing kl_coef to 0.01 and 0.1. This controls the penalty on the policy for diverging from the reference policy. A higher value (0.1) would correspond to a higher penalty and restrict the policy from drifting while a lower value (0.01) would allow the model to explore and potentially discover reward signals that reference model was not aware of. A lower kl_coef = 0.01 resulted in higher final training reward while a higher kl_coef = 0.1 resulted in a lower final training reward though eval reward was the same.

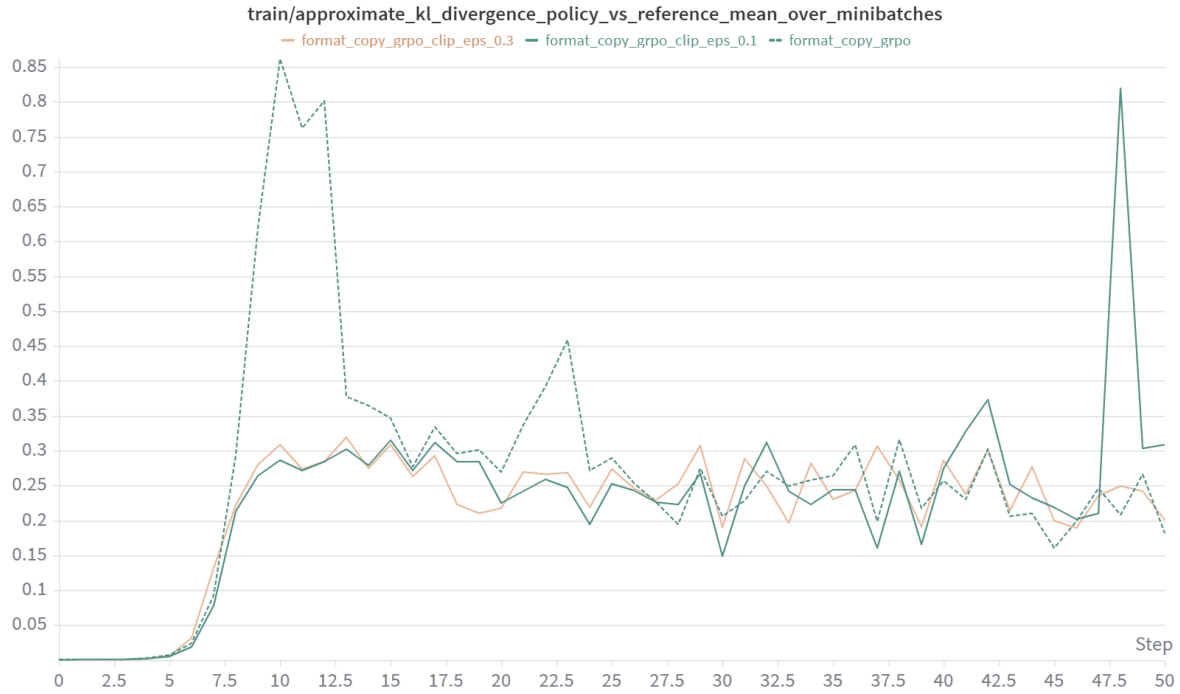
The fraction of tokens where PPO ratio was clipped was noticeably smaller for the run with kl_coef = 0.1, which makes sense because the high penalty from the high kl_coef likely soft limited how much the current policy would vary from the reference policy.

The KL_divergence of the run with kl_coef = 0.1 generally is higher than the other two runs. This may be because the high kl_coef dominates the loss term, leading to unstable updates which measured step-wise, produces sharp peaks in KL_divergence between the current and reference policy. On the other hand, the run with kl_coef = 0.01 is similar to the original run with kl_coef = 0.05 as the loss is guided by rewards which are clipped, hence smoother on average, resulting in more stable updates and a lower KL_divergence.

clip_eps Ablation







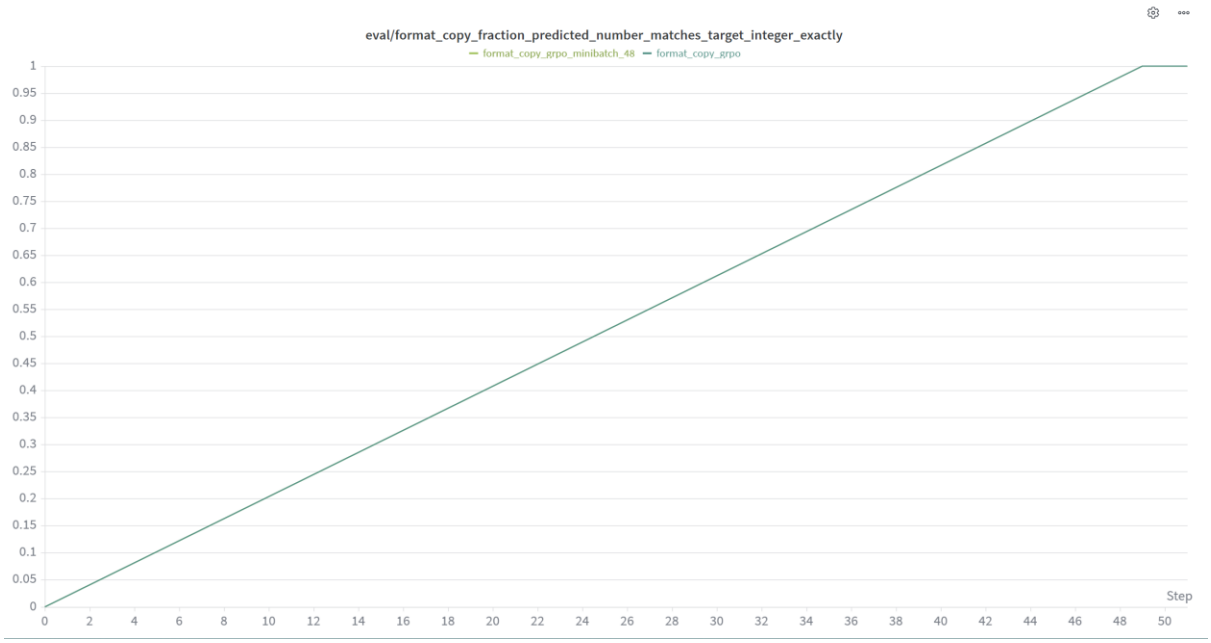
I ran format_copy grpo 2 other times, keeping other hyperparameters the same but changing clip_eps to 0.1 and 0.3. clip_eps controls the policy per_token_update. It limits the range of values that the policy per token update can take from [1- clip_eps,1+clip_eps]. This restricts the policy per token update so that the probability of tokens is not increased or decreased drastically. A higher clip_eps results in a larger range of values for the policy per token update and potentially faster learning at the risk of less stability. Eval

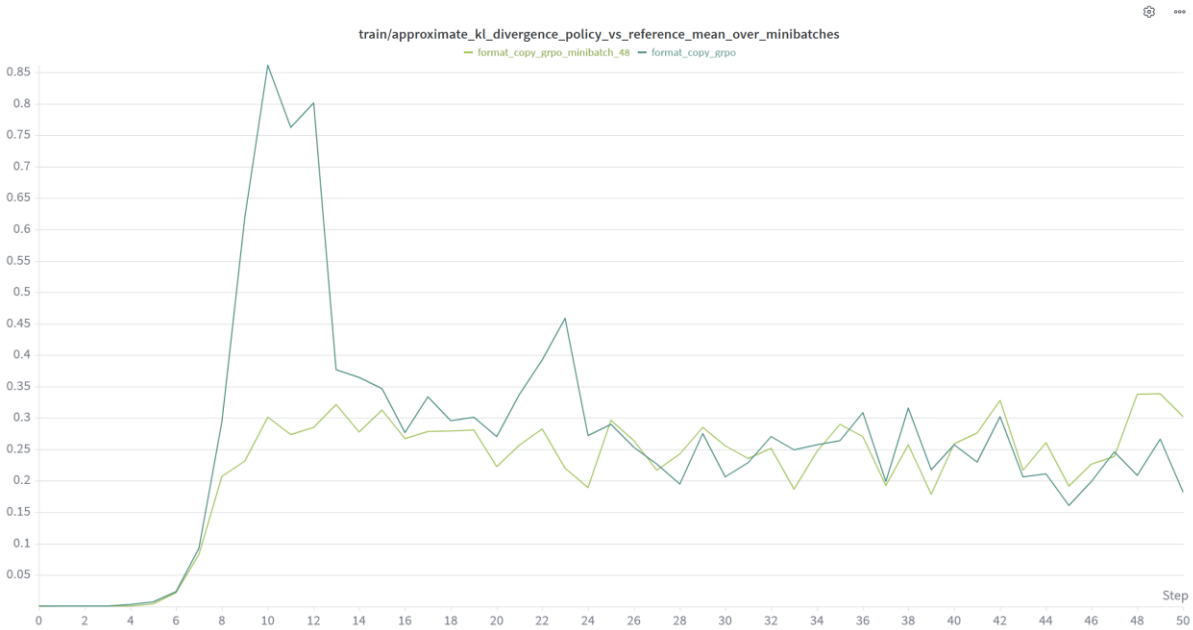
reward remained the same but train reward increased for the other two runs where clip_eps was changed.

Naturally, the run with higher clip_eps has less fraction of completion tokens clipped because there is a larger allowed range of values for the PPO per policy token update.

The KL Divergence curve is more stable for higher clip_eps potentially due to less clipping of PPO token updates resulting in a smoother gradient. On the other hand, with a smaller clip_eps, clipped PPO token updates might be uneven as some tokens would get clipped while others aren't, making the overall PPO update less smooth.

format_copy (grad_accum_steps = 1, minibatch_size = 48)





I ran format_copy grpo 1 other time, keeping other hyperparameters the same but changing grad_accum steps to 1 and minibatch size to 48. Since effective batch size = grad_accum_steps * minibatch_size, with the original grad_accum_steps = 6 and minibatch_size = 8, this maintains the effective batch size at 48 for one optimizer step. Train reward decreased in general though Eval reward stayed the same.

Maintaining group_size = 6, increasing minibatch size to 48 effectively groups together the completions for 6 distinct completions and computes the gradient over these completions. Since advantages are clipped, the mixing of gradients from different completions reduces variance. This could translate into smoother optimizer steps, leading to less KL_divergence. The smaller values of fraction of completion tokens where ppo ratio was clipped indicate that

calculating the gradient over single batch of all completions leads to less extreme per token updates.

5. **Qualitative behavior.** Include one or two model-generation examples from WandB that you found informative or surprising from the math-hard task.

math_hard_reinforce

To solve the problem, we need to find the area of triangle \(G_1 G_2 G_3\) where \(G_1\), \(G_2\), and \(G_3\) are the centroids of triangles \(PBC\), \(PCA\), and \(PAB\) respectively. Given that the area of triangle \(ABC\) is 18, we can use the properties of centroids and similar triangles to find the area of triangle \(G_1 G_2 G_3\).

Here are the steps:

1. **Understanding the Centroids**: The centroids \(G_1\), \(G_2\), and \(G_3\) divide the medians of triangles \(PBC\), \(PCA\), and \(PAB\) respectively in the ratio 2:1. This means that the triangle \(G_1 G_2 G_3\) is similar to triangle \(ABC\) with a similarity ratio of \(\frac{1}{3}\).

2. **Area Calculation**: Since the similarity ratio is \(\frac{1}{3}\), the area ratio of triangle \(G_1 G_2 G_3\) to triangle \(ABC\) is \(\left(\frac{1}{3}\right)^2 = \frac{1}{9}\). Therefore, the area of triangle \(G_1 G_2 G_3\) is \(\frac{1}{9} \times 18 = 2\).

Let's confirm this with a simple Python code using sympy.

```python
import sympy as sp

# Given area of triangle ABC
area_ABC = 18

# Similarity ratio of centroids
ratio = 1/3
```

```
# Area ratio of triangle G1G2G3 to triangle ABC
area_ratio = ratio**2

# Area of triangle G1G2G3
area_G1G2G3 = area_ratio * area_ABC

print(area_G1G2G3)
```

```output
2.0
```

The area of triangle \(G_1 G_2 G_3\) is \(\boxed{2}\).

Unlike the other completions, I found it interesting that this completion involved use of python code to verify the math calculation, whereas other completions simply performed the calculations manually.